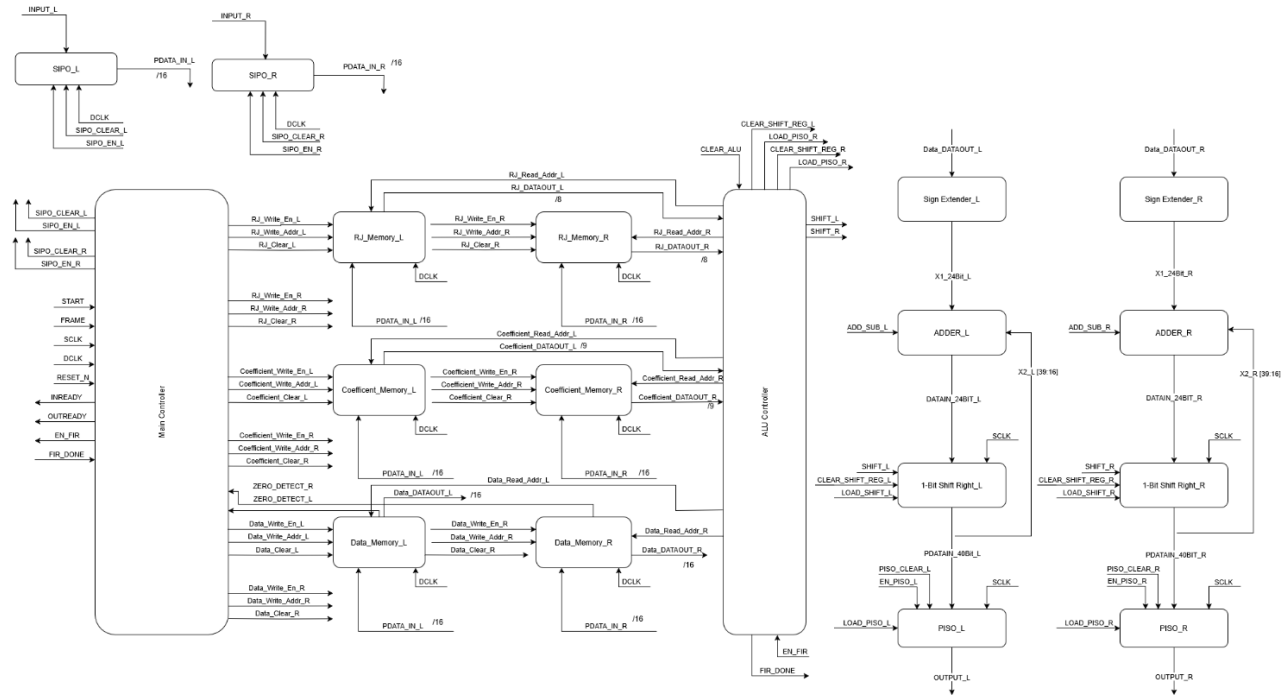
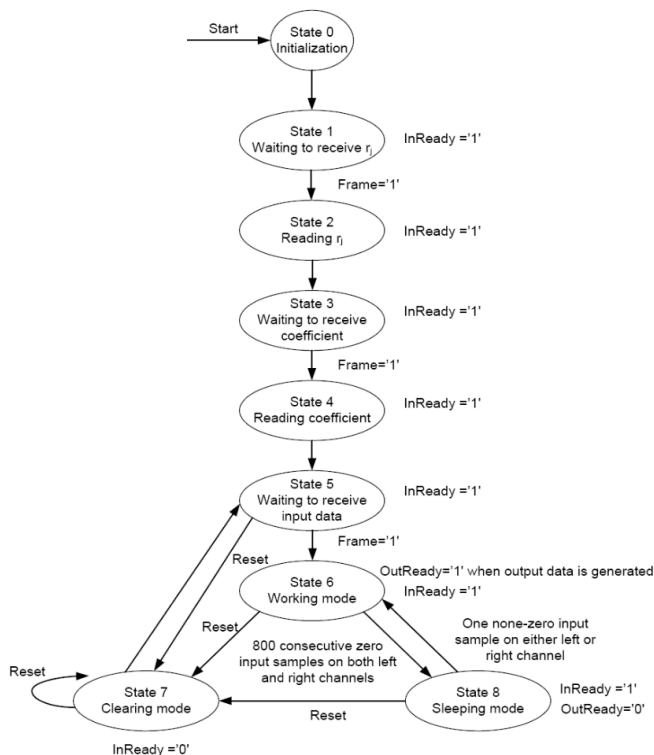


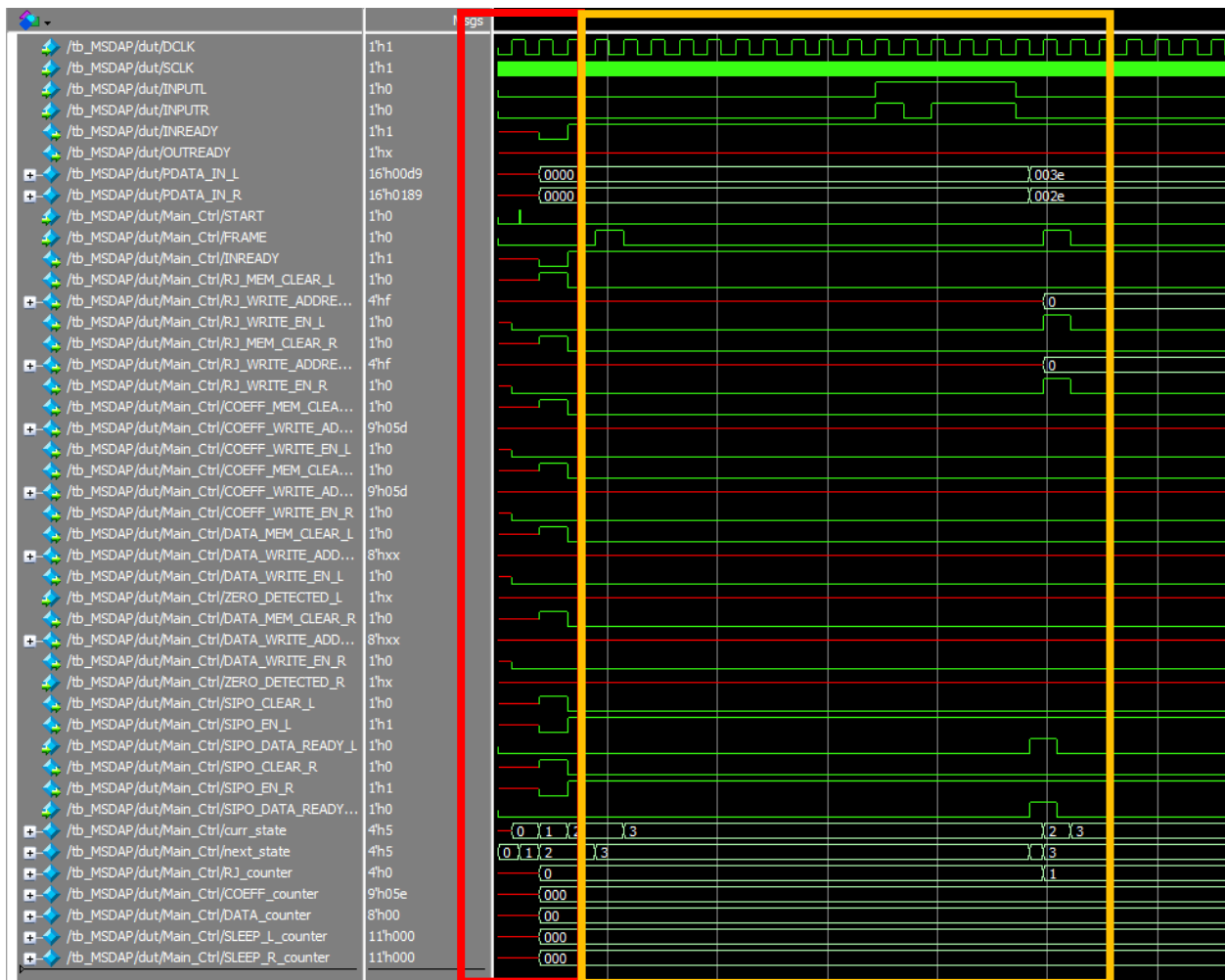
Top-Level Architecture Diagram:



The Top-Level Architecture Diagram shows how each module will be interconnected. The modules will be expanded upon in their own sections.



Main Controller: The main controller follows the Finite State Diagram outlined in the homework. See left. The code attached with this assignment is the MSDAP controller coded with the proper registers and FSM functions. Currently, the MSDAP does not support the RESET function within the FSM. The hope is to have that done before the April 7th HW #5 deadline. We can go through some key notes here, specifically the testbench and the transitions.



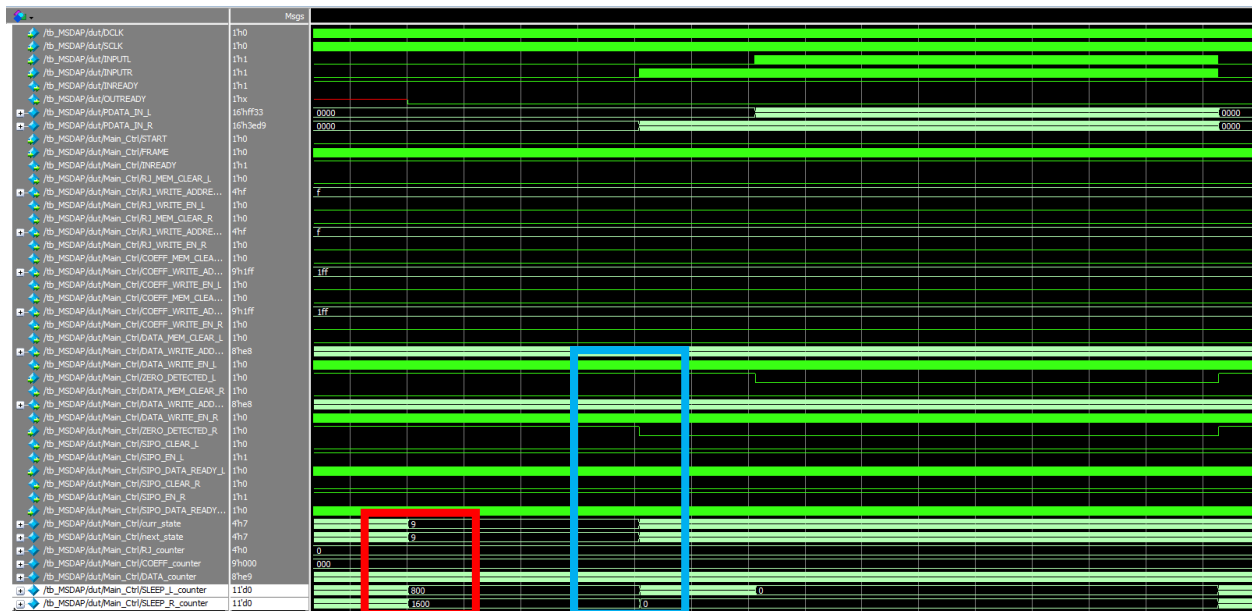
In the leftmost red block, we can see the FSM starts in state zero. Once the “START” pulse is received, the FSM transition into State 1 (Initialization). In State 1, the CLEAR signals for the MEMORIES (RJ, COEFF, and DATA), and the SIPO, go high, clearing the values within and setting everything to zero.

After State 1, the FSM immediately goes into State 2. During state two, “INREADY” is set high, signifying that the MSDAP is ready to receive data, and the testbench sends the first “FRAME” signal. After 16 cycles, the SIPO asserts that DATA is ready through DATA_READY_L/R and the controller shifts into State 3, in which it instructs the RJ memory to store at the address (the RJ counter).

This process is followed by State 4, State 5, and State 6 and State 7. A zoomed-out version of the waveform will show the COEFF and DATA reading being handled by the controller. The red box shows RJ values being stored into memory, the orange box shows COEFF values being stored into memory, and the blue box shows DATA values being stored into memory.

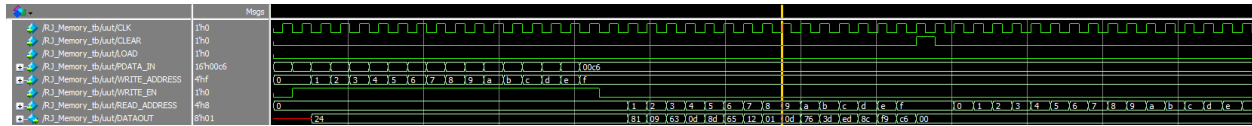


Next, the SLEEP mode of the FSM will be demonstrated. While in State 7, after 800 consecutive zero DATA values on both the left and right channel, the device will transition to State 9 (SLEEPING). You will see that after both counters reach values greater than 800, the FSM shifts to State 9 (red rectangle). Once a valid input on either L/R channel is received, the FSM shifts back into state 7 (blue rectangle).



This is as far as the Main Controller has been coded. Efforts to implement reset and clearing are underway. Next, we will look at the memories.

RJ_Memory: The RJ_Memory blocks were relatively straightforward to develop. Below is the testbench waveform.



There are three steps to this testbench. A complete write of random 8-bit data to all 16 registers, a complete read of all 16 registers, and a clear and sequential read to ensure that all registers are zero.

```
VSIM 68> run
# Starting RJ_Memory Testbench
#
# Validating memory contents after write...
#
# Validating memory contents after clear...
#
# RJ_Memory Testbench PASSED.
# ** Note: $finish : C:/Users/TheMe/Desktop/UT Dallas/EEDG_6306/Midterm/MSDAP/RJ_Memory_TB.v(101)
# Time: 64872 ns Iteration: 1 Instance: /RJ_Memory_tb
# 1
```

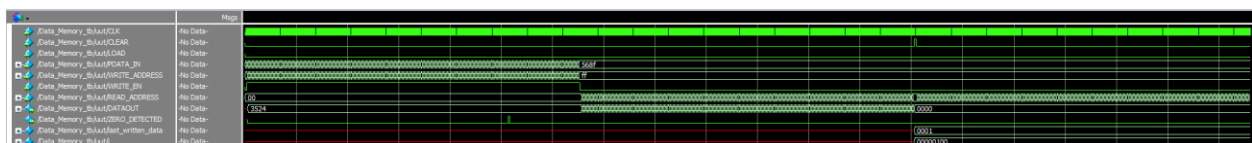
Above is the output of the console showing that the module has passed the testbench.

Coefficient_Memory: This block had the same skeleton as the RJ_Memory block; however, this block has 512 registers and each register is 9 bits in width. Console output below.



```
# Starting Coefficient_Memory Testbench
#
# Validating memory contents after write...
#
# Validating memory contents after clear...
#
# Coefficient_Memory Testbench PASSED.
# ** Note: $finish : C:/Users/TheMe/Desktop/UT Dallas/EEDG_6306/Midterm/MSDAP/Coefficient_Memory_TB.v(101)
# Time: 1957608 ns Iteration: 1 Instance: /Coefficient_Memory_tb
# 1
```

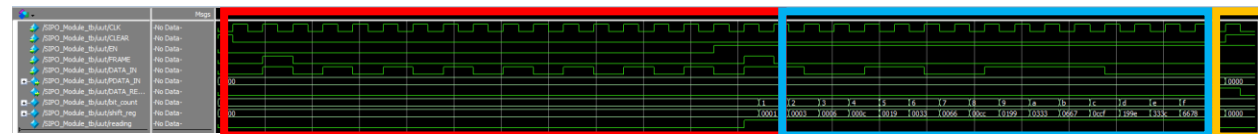
Data_Memory: Again, follows the same conventions of RJ_Memory and Coefficient_Memory with different registers and bit-width (256 and 16-bits). The Data_Memory block also features a ZERO_DETECTED function, which lets the controller know when a zero value has been received from the ADC.



	Msgs	
/Data_Memory_tb/iut/CLK	1h1	
/Data_Memory_tb/iut/CLEAR	1h0	
/Data_Memory_tb/iut/LOAD	1h0	
/Data_Memory_tb/iut/PDATA_IN	16h59f5	0000 9186 9650
/Data_Memory_tb/iut/WRITE_ADDRESS	8hcb	c8 c9 ca
/Data_Memory_tb/iut/WRITE_EN	1h1	
/Data_Memory_tb/iut/READ_ADDRESS	8h00	00
/Data_Memory_tb/iut/DATAOUT	16h3524	3524
/Data_Memory_tb/iut/ZERO_DETECTED	1h0	
/Data_Memory_tb/iut/last_written_data	16hxxxx	
/Data_Memory_tb/iut/i	32hxxxxxxxx	

```
# Starting Data_Memory Testbench
#
# Validating memory contents after write...
#
# Validating memory contents after clear...
#
# Data_Memory Testbench PASSED.
# ** Note: $finish      : C:/Users/TheMe/Desktop/UT_Dallas/EEDG_6306/Midterm/MSDAP/Data_Memory_TB.v(106)
#      Time: 980712 ns  Iteration: 1   Instance: /Data_Memory_tb
# 1
```

Below is the output waveform for the testbench.



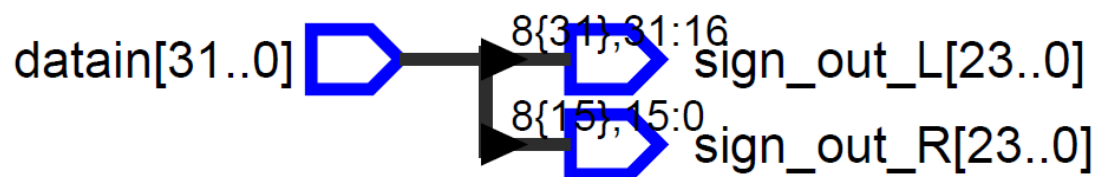
```
# ===== Starting SIPO_Module Testbench =====
# [20988000 ns] Test 1: Attempted shift with EN=0, PDATA_IN = 0000000000000000
# [42612000 ns] PDATA_IN = 1100110011110000, data_ready = 1
# PASS: Correct output and data_ready asserted
# PASS: Output cleared correctly
# ===== SIPO_Module Testbench Completed =====
# ** Note: $finish      : C:/Users/TheMe/Desktop/UT Dallas/EEDG_6306/Midterm/MSDAP/SIPO_TB.v(95)
#      Time: 43884 ns  Iteration: 0   Instance: /SIPO_Module_tb
# 1
```

SignExtender Block:

This block will take in datain sample of 32 bits and split it into 2 16bits before performing a sign extension of 8 bits base on the MSB (rightmost bit of each half). The final outputs will be sign_out_L and sign_out_R each of which s 24 bits.

<https://edaplayground.com/x/Wm3Q>

Block diagram of SignExtender: shows datain[31:0] as input and sign_out_L/R [23:0]



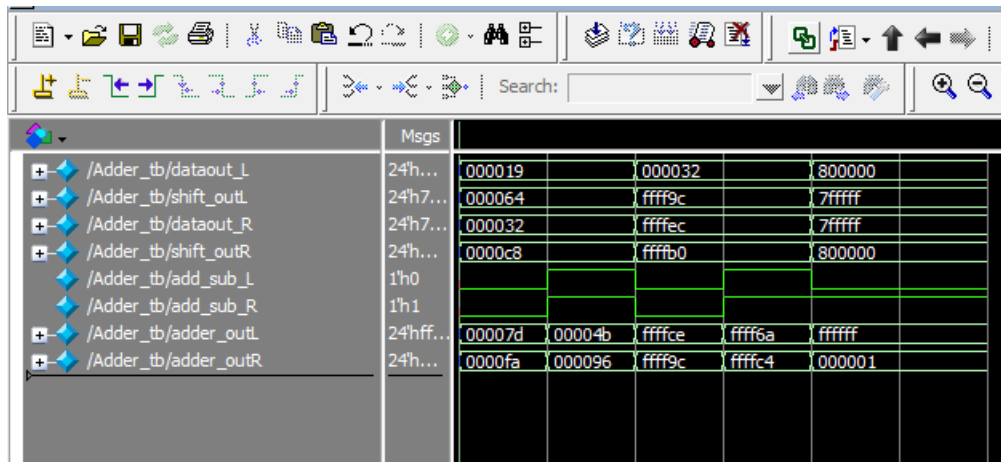
TestBench output of the SignExtender Block it splits datain two L and R

```
# run -all
# Starting SignExtender Testbench...
# Time=0 | datain=0x4bc812a4 | sign_out_L=0x004bc8 | sign_out_R=0x0012a4
# Time=10000 | datain=0x910f368d | sign_out_L=0xff910f | sign_out_R=0x00368d
# Time=20000 | datain=0x1eddafb9 | sign_out_L=0x001edd | sign_out_R=0xffafb9
# Time=30000 | datain=0x80000000 | sign_out_L=0xff8000 | sign_out_R=0x000000
# Time=40000 | datain=0x7fff7fff | sign_out_L=0x007fff | sign_out_R=0x007fff
# Time=50000 | datain=0xffff0000 | sign_out_L=0xffffffff | sign_out_R=0x000000
# Time=60000 | datain=0x0000ffff | sign_out_L=0x000000 | sign_out_R=0xffffffff
# Time=70000 | datain=0x1234abcd | sign_out_L=0x001234 | sign_out_R=0xffabcd
# Testbench finished.
```

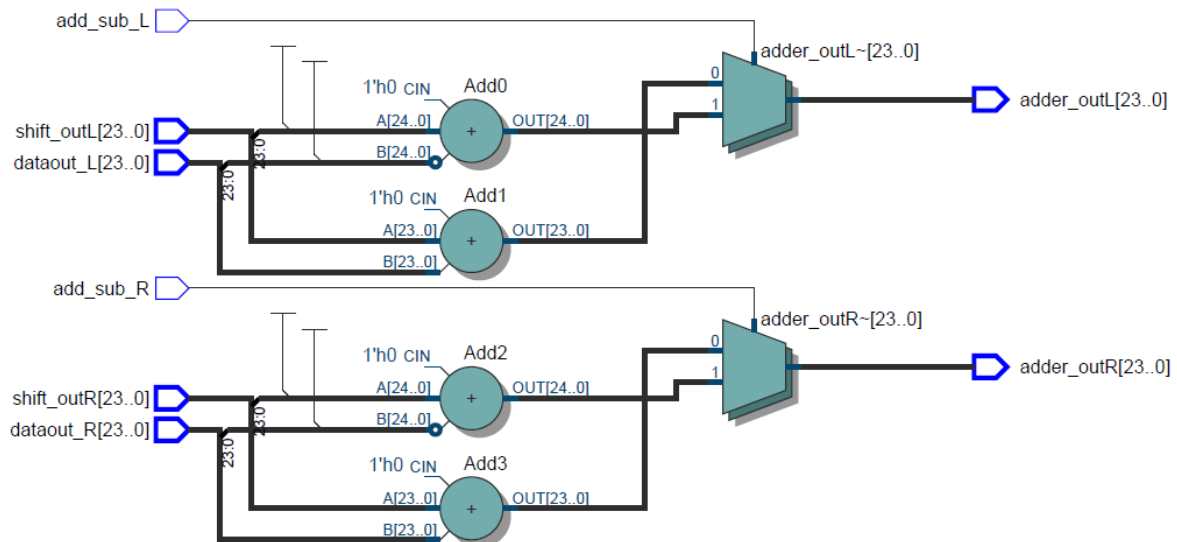
Adder block

This is two block adder meaning it add/subtract on both the right and Left side independently. The Block will take input from the SignExtender and from the 1-bitShiftRegister. It will use the MSB of the Coefficient at that stage of the filter to determine whether to perform addition or subtraction. The Block also has two independent outputs adder_outL/R

WaveView: showing sample for both addition and subtraction



Block Diagram of Adder: shows there are two blocks inside the Adder one for the right computation and one for the Left computation



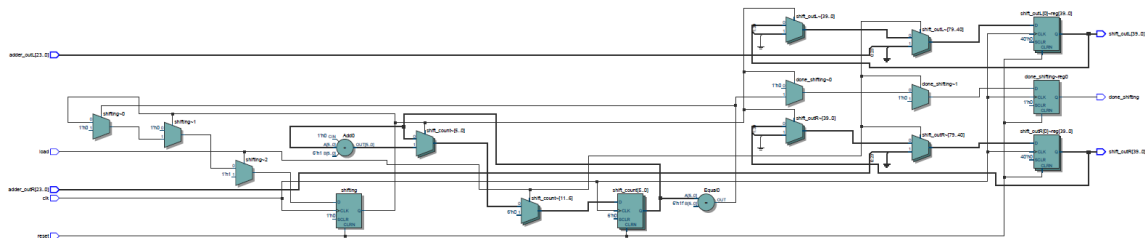
Testbench table: showing values in decimal for easy recognition

```
# vsim -t lps -L altera_ver -L lpm_ver -L sgate_ver -L altera_mf_ver -L altera_insim_ver -L cyclonev_ver -L cyclonev_hssi_ver -L cyclonev_pcie_hip_ver -L rtl_work -L work -voptargs="+acc" Adder_tb
# vsim -t lps -L altera_ver -L lpm_ver -L sgate_ver -L altera_mf_ver -L altera_insim_ver -L cyclonev_ver -L cyclonev_hssi_ver -L cyclonev_pcie_hip_ver -L rtl_work -L work -voptargs="+acc" Adder_tb
# Start time: 23:55:15 on Mar 26, 2025
# ** Note: (vsim-3012) Design is being optimized...
# Loading work.Adder_tb(fast)
# Loading work.Adder(fast)
#
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
Starting Adder Testbench...
Time=0 | L: 100 + 25 = 125 | R: 200 + 50 = 250
Time=10000 | L: 100 - 25 = 75 | R: 200 - 50 = 150
Time=20000 | L: -100 + 50 = -50 | R: -80 + -20 = -100
Time=30000 | L: -100 - 50 = -150 | R: -80 - -20 = -60
Time=40000 | L: 8388607 + -8388608 = -1 | R: -8388608 - 8388607 = 1
Testbench finished.
# ** Note: $finish : C:/Users/lmnmj/OneDrive/Documents/Grad School/UID/Spring 2025/CE 6306 ASIC Design/Projects/Midterm/ALU_Block/Adder/Adder_tb.v(74)
# Time: 60 ns Iteration: 0 Instance: /Adder_tb
# 1
```

ShiftRightRegister

This is a 1 bit shifter, it takes the output of each side of the adder and shifts it to the right by 1-bit, and sends the results back to the adder where the result will be accumulated with the new datain sample. This process repeats until the convolution is completed, then the final result is sent to the PISO block.

Block Diagram of ShiftRightRegister:



WaveView of the Simulated Testbench

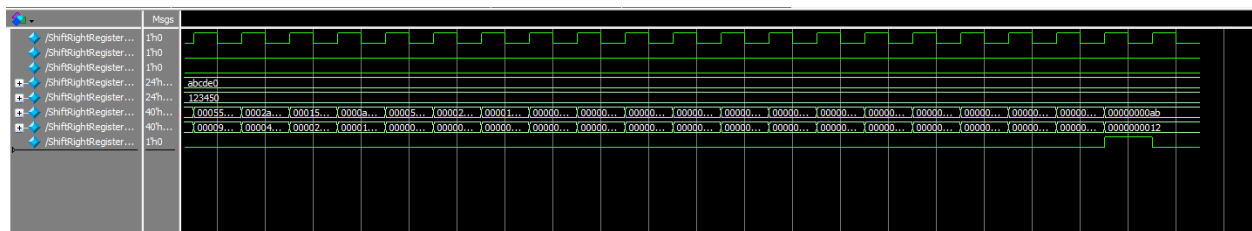


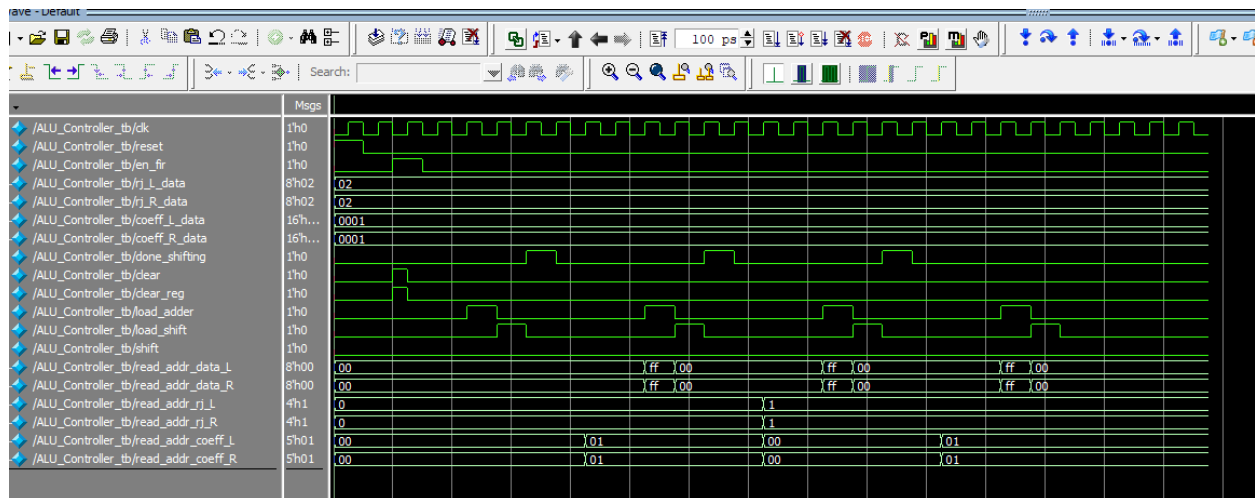
Table showing the simulated result

```
# run -all
# T=0 | load=0 | done=0 | shift_outL=0x0000000000 | shift_outR=0x0000000000
# T=20000 | load=1 | done=0 | shift_outL=0x0000000000 | shift_outR=0x0000000000
# T=25000 | load=1 | done=0 | shift_outL=0xabcd000000 | shift_outR=0x1234500000
# T=30000 | load=0 | done=0 | shift_outL=0xabcd000000 | shift_outR=0x1234500000
# T=35000 | load=0 | done=0 | shift_outL=0x5566f00000 | shift_outR=0x091a280000
# T=40000 | load=0 | done=0 | shift_outL=0x2af3780000 | shift_outR=0x048d140000
# T=45000 | load=0 | done=0 | shift_outL=0x1579bc0000 | shift_outR=0x02468a0000
# T=50000 | load=0 | done=0 | shift_outL=0xabcd000000 | shift_outR=0x0123450000
# T=55000 | load=0 | done=0 | shift_outL=0x0556f00000 | shift_outR=0x0091a28000
# T=60000 | load=0 | done=0 | shift_outL=0x02af378000 | shift_outR=0x0048d14000
# T=65000 | load=0 | done=0 | shift_outL=0x01579bc000 | shift_outR=0x002468a000
# T=70000 | load=0 | done=0 | shift_outL=0x0000abcd00 | shift_outR=0x0001234500
# T=75000 | load=0 | done=0 | shift_outL=0x00005566f0 | shift_outR=0x0000091a28
# T=80000 | load=0 | done=0 | shift_outL=0x00002af378 | shift_outR=0x0000048d14
# T=85000 | load=0 | done=0 | shift_outL=0x00001579bc | shift_outR=0x000002468a
# T=90000 | load=0 | done=0 | shift_outL=0x000000abcd | shift_outR=0x0000001234
# T=95000 | load=0 | done=0 | shift_outL=0x0000005566f | shift_outR=0x000000091a
# T=100000 | load=0 | done=0 | shift_outL=0x0000002af37 | shift_outR=0x000000048d
# T=105000 | load=0 | done=0 | shift_outL=0x0000001579b | shift_outR=0x00000002468
# T=110000 | load=0 | done=0 | shift_outL=0x00000000abcd | shift_outR=0x000000001234
# T=115000 | load=0 | done=0 | shift_outL=0x000000005566f | shift_outR=0x00000000091a
# T=120000 | load=0 | done=0 | shift_outL=0x000000002af37 | shift_outR=0x00000000048d
# T=125000 | load=0 | done=0 | shift_outL=0x000000001579b | shift_outR=0x0000000002468
# T=130000 | load=0 | done=0 | shift_outL=0x0000000000abcd | shift_outR=0x00000000001234
# T=135000 | load=0 | done=0 | shift_outL=0x00000000005566f | shift_outR=0x0000000000091a
# T=140000 | load=0 | done=0 | shift_outL=0x00000000002af37 | shift_outR=0x0000000000048d
# T=145000 | load=0 | done=0 | shift_outL=0x00000000001579b | shift_outR=0x000000000002468
# T=150000 | load=0 | done=0 | shift_outL=0x0000000000abcd | shift_outR=0x000000000001234
# T=155000 | load=0 | done=0 | shift_outL=0x00000000005566f | shift_outR=0x00000000000091a
# T=160000 | load=0 | done=0 | shift_outL=0x00000000002af37 | shift_outR=0x00000000000048d
# T=165000 | load=0 | done=0 | shift_outL=0x00000000001579b | shift_outR=0x0000000000002468
# T=170000 | load=0 | done=0 | shift_outL=0x000000000000abcd | shift_outR=0x0000000000001234
# T=175000 | load=0 | done=0 | shift_outL=0x0000000000005566f | shift_outR=0x000000000000091a
# T=180000 | load=0 | done=0 | shift_outL=0x0000000000002af37 | shift_outR=0x000000000000048d
# T=185000 | load=0 | done=0 | shift_outL=0x0000000000001579b | shift_outR=0x00000000000002468
# T=190000 | load=0 | done=0 | shift_outL=0x000000000000abcd | shift_outR=0x00000000000001234
# T=195000 | load=0 | done=0 | shift_outL=0x0000000000005566f | shift_outR=0x0000000000000091a
# T=200000 | load=0 | done=0 | shift_outL=0x0000000000002af37 | shift_outR=0x0000000000000048d
# T=205000 | load=0 | done=0 | shift_outL=0x0000000000001579b | shift_outR=0x000000000000002468
# T=210000 | load=0 | done=0 | shift_outL=0x00000000000000abcd | shift_outR=0x000000000000001234
# T=215000 | load=0 | done=0 | shift_outL=0x000000000000005566f | shift_outR=0x00000000000000091a
# T=220000 | load=0 | done=0 | shift_outL=0x000000000000002af37 | shift_outR=0x00000000000000048d
# T=225000 | load=0 | done=0 | shift_outL=0x000000000000001579b | shift_outR=0x0000000000000002468
# T=230000 | load=0 | done=0 | shift_outL=0x0000000000000000abcd | shift_outR=0x00000000000000001234
# T=235000 | load=0 | done=0 | shift_outL=0x00000000000000005566f | shift_outR=0x0000000000000000091a
# T=240000 | load=0 | done=0 | shift_outL=0x00000000000000002af37 | shift_outR=0x0000000000000000048d
# T=245000 | load=0 | done=0 | shift_outL=0x00000000000000001579b | shift_outR=0x000000000000000002468
# T=250000 | load=0 | done=0 | shift_outL=0x0000000000000000abcd | shift_outR=0x000000000000000001234
# T=255000 | load=0 | done=0 | shift_outL=0x00000000000000005566f | shift_outR=0x00000000000000000091a
# T=260000 | load=0 | done=0 | shift_outL=0x00000000000000002af37 | shift_outR=0x00000000000000000048d
# T=265000 | load=0 | done=0 | shift_outL=0x00000000000000001579b | shift_outR=0x0000000000000000002468
# T=270000 | load=0 | done=0 | shift_outL=0x000000000000000000abcd | shift_outR=0x0000000000000000001234
# T=275000 | load=0 | done=0 | shift_outL=0x0000000000000000005566f | shift_outR=0x000000000000000000091a
# T=280000 | load=0 | done=0 | shift_outL=0x0000000000000000002af37 | shift_outR=0x000000000000000000048d
# T=285000 | load=0 | done=0 | shift_outL=0x0000000000000000001579b | shift_outR=0x00000000000000000002468
# T=290000 | load=0 | done=0 | shift_outL=0x00000000000000000000abcd | shift_outR=0x000000000000000000001234
# T=295000 | load=0 | done=0 | shift_outL=0x000000000000000000005566f | shift_outR=0x00000000000000000000091a
# T=300000 | load=0 | done=0 | shift_outL=0x000000000000000000002af37 | shift_outR=0x00000000000000000000048d
# T=305000 | load=0 | done=0 | shift_outL=0x000000000000000000001579b | shift_outR=0x0000000000000000000002468
# T=310000 | load=0 | done=0 | shift_outL=0x00000000000000000000abcd | shift_outR=0x0000000000000000000001234
# T=315000 | load=0 | done=0 | shift_outL=0x000000000000000000005566f | shift_outR=0x000000000000000000000091a
# T=320000 | load=0 | done=0 | shift_outL=0x000000000000000000002af37 | shift_outR=0x000000000000000000000048d
# T=325000 | load=0 | done=0 | shift_outL=0x000000000000000000001579b | shift_outR=0x00000000000000000000002468
# T=330000 | load=0 | done=0 | shift_outL=0x00000000000000000000abcd | shift_outR=0x00000000000000000000001234
# Shifting done at time 345000
# Final shift_outL = 0x0000000000ab
# Final shift_outR = 0x000000000012
# T=345000 | load=0 | done=1 | shift_outL=0x0000000000ab | shift_outR=0x000000000012
# T=355000 | load=0 | done=0 | shift_outL=0x0000000000ab | shift_outR=0x000000000012
# ** Note: $finish : C:/Users/lanjm/OneDrive/Documents/Grad School/UTD/Spring 2025/CE 6306 ASIC Design/Projects/Midterm/ALU_Block/ShiftRightRegister/ShiftRightRegister_tb.v(49)
# Time: 365 ns Iteration: 0 Instance: /ShiftRightRegister_cb
```

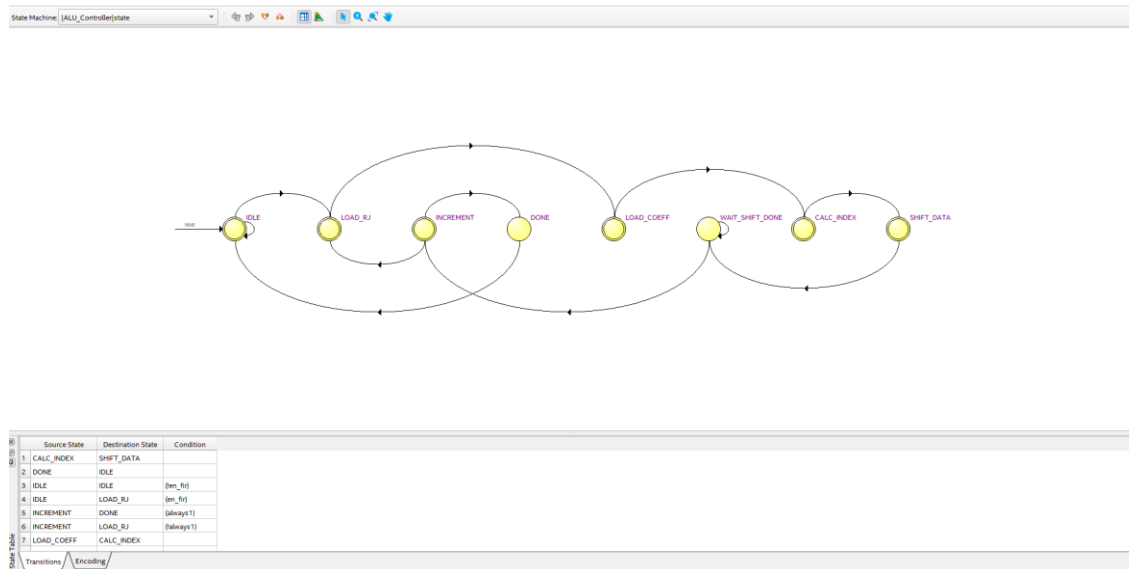
ALU_Controller Block

This controls the interaction between the ALU and other Blocks inside the MSDAP. It also controls the local behavior of the ALU block meaning how the internal blocks inside the ALU interact with each other in terms of shifting and computation. The main challenge in making this block is figuring out when to bring in datain sample or which address of the data_memory to use. An index was created to compute the address of the data_memory to be used at each stage of the convolution. This block is the most challenging part of the ALU block.

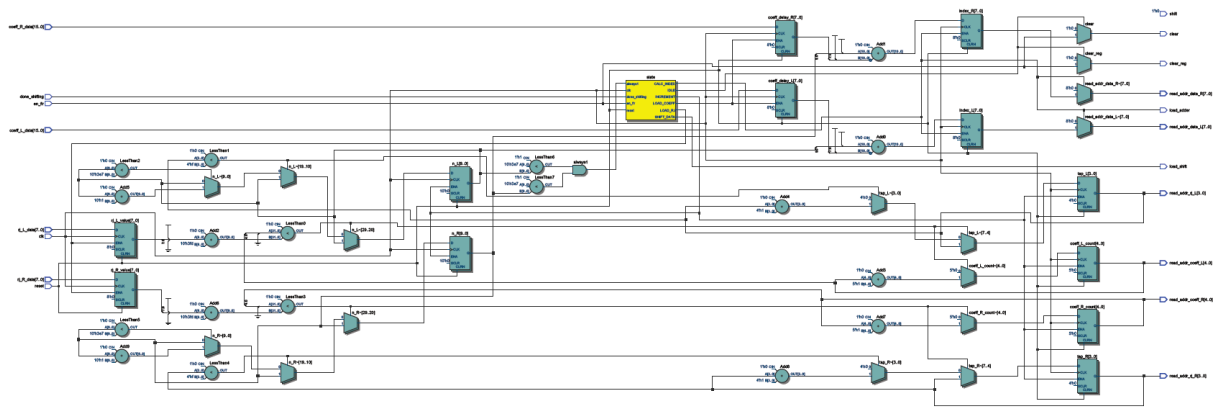
WaveView after the Simulation



State Diagram of the ALU Finite State Machine



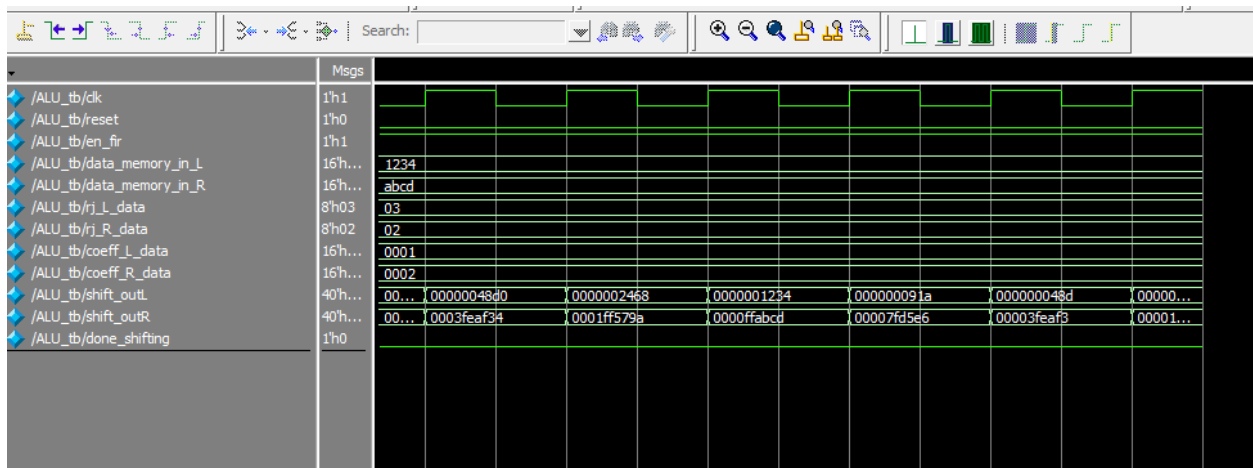
ALU Block diagram: the yellow is the FSM block of the ALU controller



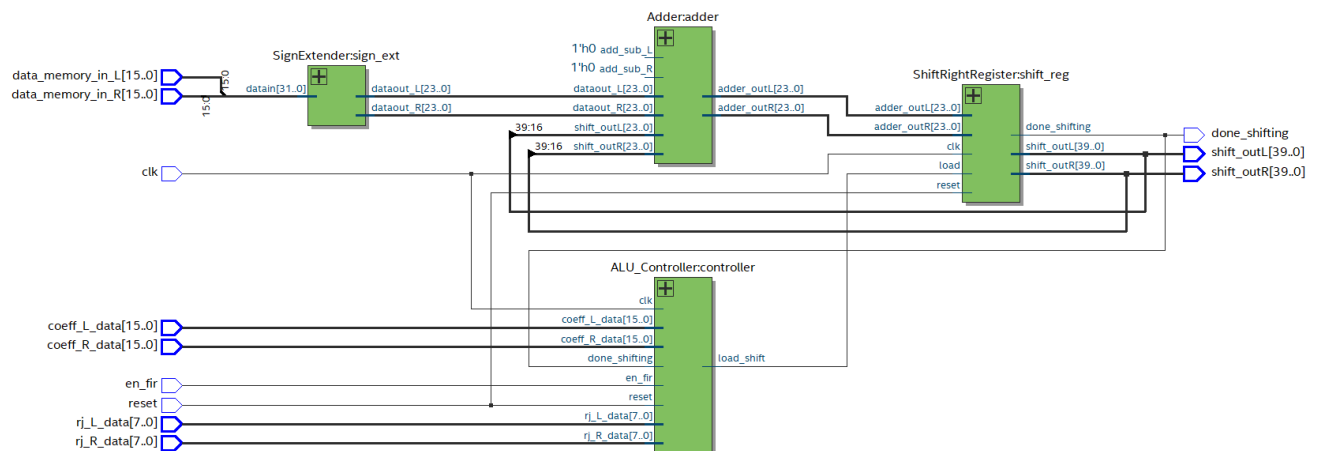
ALU_Top Module

This Block connects all the submodules inside the ALU. Each submodule is connected using the ports name instead of the port order to avoid mismatch of port connections.

WaveView of the ALU_Top Module



Block Diagram of the ALU Top Module



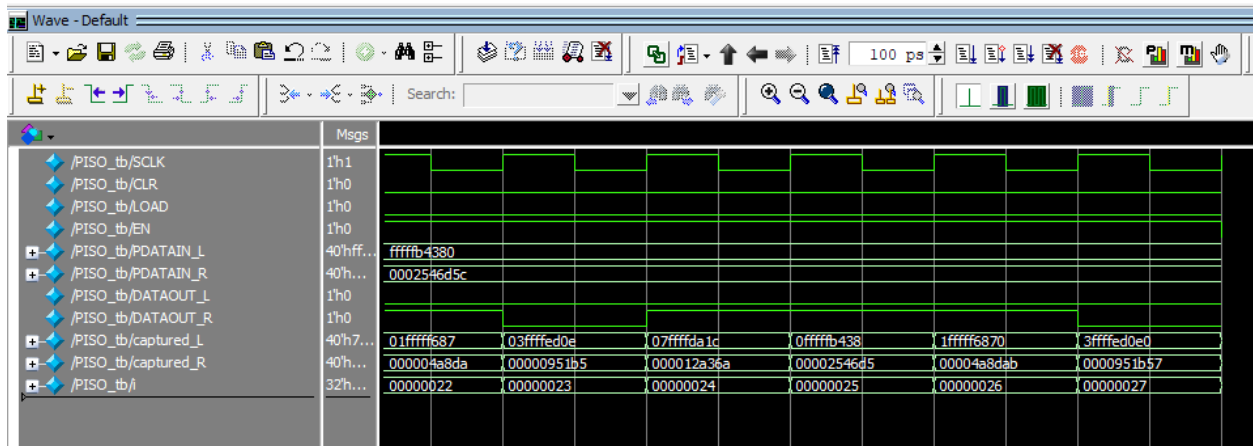
Simulated Output for the ALU_Top

```
# vsim -t ips -L altera_ver -L ipm_ver -L sgate_ver -L altera_mr_ver -L altera_insim_ver -L cyclonev_ver -L cyclonev_nssi_ver -L cyclonev_pcie_nlp_ver -L rtl_work -L work
# Start time: 23:51:12 on Mar 26, 2025
# ** Note: (vsim-3812) Design is being optimized...
# ** Note: (vopt-143) Recognized 1 FSM in module "ALU_Controller(fast)".
# Loading work.ALU_tb(fast)
# Loading work.ALU(fast)
# Loading work.SignExtender(fast)
# Loading work.Adder(fast)
# Loading work.ShiftRightRegister(fast)
# Loading work.ALU_Controller(fast)
#
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
# Shift Output L: 0000000246
# Shift Output R: 00001ff579
# ** Note: $stop : C:/Users/lmnjm/OneDrive/Documents/Grad School/UTD/Spring 2025/CE 6306 ASIC Design/Projects/Midterm/ALU_Block/ALU_Top_Module/ALU_tb.v(54)
# Time: 1020 ns Iteration: 0 Instance: /ALU_tb
# Break in Module ALU_tb at C:/Users/lmnjm/OneDrive/Documents/Grad School/UTD/Spring 2025/CE 6306 ASIC Design/Projects/Midterm/ALU_Block/ALU_Top_Module/ALU_tb.v line 54
```

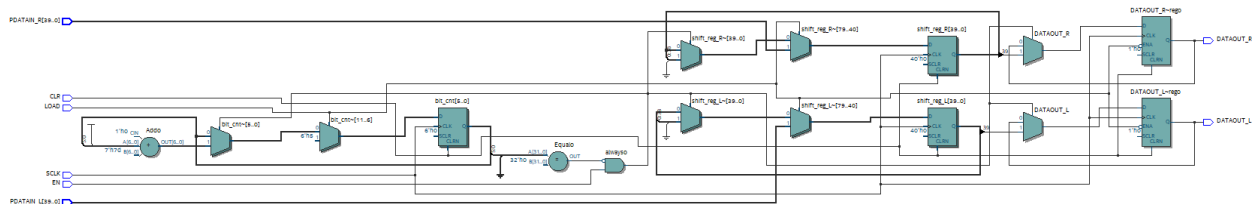
PISO_BLOCK

This Block takes the convoluted output from the ALU block in parallel form all bits at once and stores it into an internal register. After the output is stored in the internal register of the PISO block. The data is shifted left one bit at a time (serially starting with the MSB). The most challenging part of this block is managing the bits to make sure `dataout[0]` does not have some of its bits mixed with `dataout[1]`.

WaveView of the testbench



Block Diagram of the PISO block



Predicted MSDAP Performance: Although we don't have a top-level design of the MSDAP, we can make some assumptions about reading the data in through our initial testbench.

To store all of the RJ values takes 330.4 micro-seconds.

To store all of the COEFFICIENT values takes 10421 micro-seconds.

Modifications to Block Level Design: No changes to the block or top level design were made.

Challenges Faced: This was our first time working with Verilog. It is a very unfriendly language to learn, and it took a long time to ramp up and become proficient enough to code, especially the finite state machine. Working through the virtual machine proved to be a pain, since it has a lag that makes navigating the GUI painful. This was mitigated by acquiring a license from Intel to run ModelSim on my personal computer.

Generative AI: Generative AI was solely used to develop a testbench for the individual blocks and to help with syntax checking. Most times, generative AI suggests a code solution that is too complex. Using it to generate code is usually not a good idea.

It is however, useful in checking syntax errors or logical errors within the code. It's fixes aren't usually the most elegant, but it being able to identify them quickly is a huge help.